

ARM Instruction Set

- 32 bit instruction set.

Name	Example	Comments
16 registers	r0,r1,r2.....r11,....r12..... sp...lr,pc	Fast locations for data,in ARM,data must be in registers to perform arithmetic.
2 ³⁰ memory words	Memory[0],memory[4]... Memory[4294967292]	ARM uses byte addresses ,so sequential address differ by 4

- Arithmetic(Add,Sub....)
- Data Transfer(load register,store register,mov etc.)
- Logical(and,or,not...)
- Conditional branch(branch on EQ,NE,LT,LE...)
- Unconditional branch(branch(always),branch and link)

MIPS Instruction Set

Name	Example	Comments
32 registers	\$s0-\$s7; \$t0-\$t9,\$zero, \$a0-\$a3, \$v0-\$v1,\$gp,\$fp,\$sp,\$ra,\$at	Fast locations for data,in MIPS,data must be in registers to perform arithmetic.register \$zero always equals 0,and register \$at is reserved by the assembler to handle large constants
2^{30} memory words	Memory[0],memory[4]... Memory[4294967292]	MIPS uses byte addresses ,so sequential address differ by 4,Memory holds DS, arrays, and spilled registers

comparison

Category	Instruction	ARM	MIPS
Arithmetic	Add Sub Add immediate	ADD r1,r2,r3 SUB r1,r2,r3	add \$s1,\$s2,\$s3 sub \$s1,\$s2,\$s3 addi \$s1,\$s2,20
Data Transfer	Load register/ Load word Store register/ store word	LDR r1,[r2,#20] {r1=memory[r2+20]} STR r1,[r2,#20]	Lw \$s1,20(\$s2) Sw \$s1,20(\$s2)

ARM assembly language notation

ARM arithmetic instruction performs only one operation and must always have exactly three variables. Ex. place the sum of four variables b,c,d,e into variable a.

- ADD a,b,c (sum of b and c placed in a)
- ADD a,a,d (sum of b,c and d is now in a)
- ADD a,a,e (sum of b,c,d, and e is now in a)

It takes three instructions to sum the four variables.

First Design Principle: Simplicity favours regularity

Requiring every instruction to have exactly three operands, no more and no less. Conforms to the philosophy of keeping the h/w simple; h/w for variable number of operands is more complicated than h/w for a fixed number.

C assignment to ARM assembly

- **Second Design Principle: smaller is faster**(A very large number of registers may increase the clock cycle time because it takes electronic signals longer when they must travel farther. smaller is faster ;15 registers may not be faster than 16 .Energy is also the major concern ,fewer registers is to conserve energy)
- $a=b+c; d=a-e$; ADD a,b,c , SUB d,a,e
- $f=(g+h)-(i+j)$; ADD t0,g,h ,ADD t1,i,j,SUB f,t0,t1.

Memory operands: for simple variables single data elements is used but for complex data structures –arrays, structure etc.The DSs can contain many more elements than there are registers in computer. How can a computer represent and access such large structures? The processor can keep only small amount of data in registers, but memory contains billions of data elements, Hence DSs(array and structures)are kept in memory.

- Compile the C assignment , $g = h + A[8]$

LDR r1,[r2,#32]

r2=base address

r1=temp register

- ADD r3,r4,r1

- Offset=8

- In ARM word must

Start at address that are

Multiples of 4

(alignment restriction)

32		8
24		7
20		6
16		5
12		4
8		3
4		2
0		1

Byte Address data element

Compile using load and store $A[12]=h+A[8]$

```
LDR r1,[r2,#32]
```

```
ADD r4,r3,r1
```

```
STR r4,[r2,#48]
```

Many programs have more variables than computers have registers. Consequently the compilers tries to keep the most frequently used variables in registers and places the rest in memory,using loads and stores to move variables between registers and memory.The process of putting less commonly used variables (or those needed later) into memory is called ***spilling registers***.

To achieve highest performance and conserve energy,compilers must use registers efficiently.

Constant or Immediate Operands

Many times a program will use a constant in an operation. Ex .increment an index to point to the next element of an array. More than half of the ARM arithmetic instructions have a constant as an operand when running the SPEC2006 benchmarks.

Add a constant 4 to register r3.

```
LDR r5,[r1,#AddrConstant4]
```

```
ADD r3,r3,r5
```

r1+AddrConstant4 is the memory address of the constant.

```
ADD r3,r3,#4 {r3=r3+4},immediate operand.
```

Third Design Principle: Make the common case fast(Constant operands occur frequently ,and by including constants inside arithmetic instructions, operations are much faster and use less energy than if constants were loaded from memory)

Signed and Unsigned Numbers



MSB=31

LSB=0

ARM word is 32 bits long.

So we can represent the numbers from 0 to $2^{32}-1$ (4294967295)

0000	0000	0000	0000	0000	0000	0000	0000=0
0000	0000	0000	0000	0000	0000	0000	0001=1
0000	0000	0000	0000	0000	0000	0000	0010=2
.....							
1111	1111	1111	1111	1111	1111	1111	1111
	=4294967295						

Positive and negative numbers represented by separate sign by single bit 0(positive) and 1(negative).

Integers representation: a)Signed Magnitude representation

b)Signed-1's complement representation

c) Signed-2's complement representation

Ex 14 in 8- bit register=0000 1110

Three different ways to represent -14=

i) Signed Magnitude representation=1 0001110

ii) Signed-1's complement representation=1 1110001

iii)Signed-2's complement representation=1 1110010

Conclusions

- Signed magnitude system is used in ordinary arithmetic but is awkward when employed in computer arithmetic.
- Hence signed complement is used ,but 1's complement impose difficulties because it has two representation of 0(+0 and -0).1's complement is used for logical operation.
- 2's complement is used for representing negative numbers.

2's complement addition: 6 00000110

13 00001101

19 00010011

Do -6+13,6-13,-6-13.

Translating ARM assembly into m/c instruction

- ADD r5,r1,r2

The decimal representation:

14	0	0	4	0	1	5	2
----	---	---	---	---	---	---	---

Fourth field tells that the instruction performs addition

Instruction in binary form:instruction format

1110	00	0	0100	0	0001	0101	000000000010
4bits	2bits	1bit	4bits	1bit	4bits	4bits	12bits

ARM Fields:

Cond	F	I	Opcode	S	Rn	Rd	Operand2
4bits	2bits	1bit	4bits	1bit	4bits	4bits	12bits

Cond: conditional branch instruction

F:Instruction format,I=immediate(if 0 the second source operand is register ,if 1 than 12 bit immediate),S=set condition(related to cond.branch)

ADD r3,r3,#4 , r3=r3+4

14	0	1	4	0	3	3	4
4bits	2bits	1bit	4bits	1bit	4bits	4bits	12bits

LDR r5,[r3,#32] ,temp reg r5 gets A[8]

Load and store instruction use 6 fields

Cond	F	Opcode	Rn	Rd	Offset2
4bits	2bits	6bit	4bits	4bit	12bits

F=1 data transfer instruction,24=load word

14	1	24	3	5	32
4bits	2bits	6bit	4bits	4bit	12bits

Instruction	Format	Cond	F	I	Op	S	Rn	Rd	Operand2
ADD	DP	14	0	0	4	0	Reg	Reg	Reg
SUB	DP	14	0	0	2	0	Reg	Reg	Reg
ADD(immediate)	DP	14	0	1	4	0	Reg	Reg	Constant
LDR	DT	14	1	Na	24	Na	Reg	Reg	Address
STR	DT	14	1	Na	25	Na	Reg	Reg	Address

A[30]=h+A[30]

LDR r5,[r3,#120]

ADD r5,r2,r5

STR r5,[r3,#120]

Cond	F	I	Op	S	Rn	Rd	Operand2
14	1		24		3	5	120
14	0	0	4	0	2	5	5
14	1		25		3	5	120

Cond	F	I	Op	S	Rn	Rd	Operand2
1110	1		11000		0011	0101	0000 1111 0000
1110	0	0	100	0	0010	0101	0000 0000 0101
1110	1		11001		0011	0101	0000 1111 0000

Logical Operations

AND r5,r1,r2,reg5=reg1 & reg2

MVN r5,r1 ,reg5=~reg r1,[move Not]

MOV r6,r5 ,reg6=reg5

MIPS Code for $f=(g+h)-(i+j)$

Assign variables to registers:\$s0=f,\$s1=g,\$s2=h,\$s3=i,\$s4=j

add \$t0,\$s1,\$s2

add \$t1,\$s3,\$s4

sub \$s0,\$t0,\$t1

For $g=h+A[8]$

lw \$t0,32(\$s3) ,tem reg t0 gets A[8]

add \$s1,\$s2,\$t0

ADD r5,r1,r2 in ARM(decimal representation)

14	0	0	4	0	1	5	2
----	---	---	---	---	---	---	---

MIPS decimal representation

add \$t0,\$s1,\$s2

17=\$s1,18=\$s2,8=\$t0,

0	17	18	8	0	32
---	----	----	---	---	----

6bits

5bits

5bits

5bits

5bits

6bits

MIPS Fields

op	rs	rt	rd	shamt	funct
6bits	5bits	5bits	5bits	5bits	6bits

op=opcode

rs=first register source operand

rt=second register source operand

rd=register destination operand

shamt=shift amount

funct=function, this field selects the specific variant of the operation in the op field also called as function code.

- The problem occurs when an instruction needs longer fields than those shown. Ex the load word instruction must specify two registers and a constant. If the address uses one of the 5 fields of the format, the constant within the load word would be limited to $2^5 = 32$. This constant is used to select elements from the arrays or data structures and is often much larger than 32.
- We have conflict between desire to keep all the instructions the same length and the desire to have a single instruction format.
- **Design principle four:** Good design demands good compromises.
- The compromise chosen is to keep all instructions the same length, thereby requiring different kinds of instruction formats for different kinds of instructions.
- R-type(register format)(as shown above)
- I-type(immediate or data transfer)

op	rs	rt	Const or address
6bits	5bits	5bits	16bits

I-format

16 bit address means load word instruction can load any word within a region of $\pm 2^{15}$ or 8192 words of address in the base register rs.

Lw \$t0 32(\$s3) temp reg t0 gets A[8].

Inst	Format	Op	Rs	Rt	Rd	Shamt	Funct	address
add	R	0	Reg	Reg	Reg	O	32	Na
sub	R	0	Reg	Reg	Reg	O	34	Na
addi	I	8	Reg	Reg	Na	Na	Na	Const
lw	I	35	Reg	Reg	Na	Na	Na	Address
sw	I	43	Reg	Reg	Na	Na	Na	Address

Translate MIPS assembly into m/c code

- $A[300] = h + A[300]$

Lw \$t0,1200(\$t1) #temp reg t0 gets A[300]

add \$t0,\$s1,\$t0 #temp reg to gets h+A[300]

sw \$t0,1200(\$t1) # stores h+A[300] back into A[300]

op	rs	rt	rd	shamt/ address	funct
35	9	8		12000	
0	18	8	8	0	32
43	9	8		1200	

Op	Rs	Rt	Rd	Shamt/address	funct
100011	01001	01000		0000 0100 1011 0000	
0	10010	01000	01000	0	100000
101011	01001	01000		0000 0100 1011 0000	

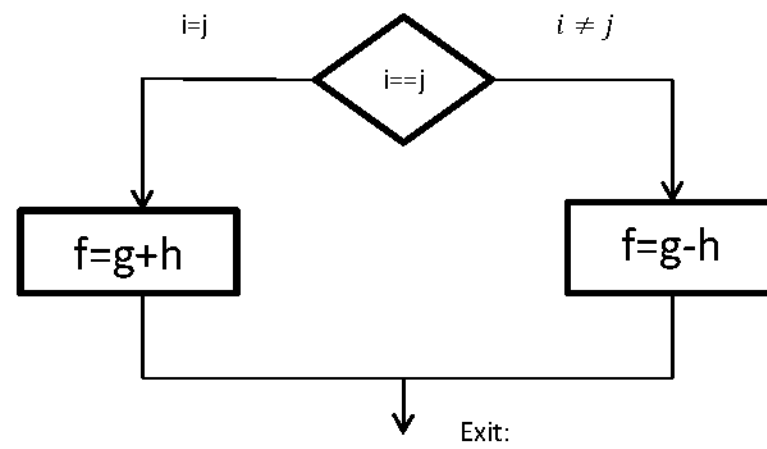
m/c code

\$s0-\$s7 mapped to 16-23

\$t0-\$t7 mapped to 8-15

- In MIPS
- Shift left logical(sll) , ex. sll \$t0,\$s0,4 # reg \$t0=\$regs0<<4 bits
- Shift right logical(srl)
- In ARM
- Logical shift left(LSL),ex LSL r5,r1 LSL #2,# r5=r1<<2
- Logical shift right(LSR),ex MOV r6,r5 LSR #4 , #r6=r5>>4
- In MIPS

op	rs	rt	rd	shamt	funct
0	0	16	10	4	0



- Shifting left by i bits gives same result as multiplying by 2^i
- Shifting right by i bits gives same result as divide by 2^i

Instructions for making decisions:in ARM

- If($i=j$), $f=g+h$;else $f=g-h$;

Assigning variables to registers: $r0=f,r1=g,r2=h,r3=i,r4=j$

CMP $r3,r4$

BNE Else;go to else if $i \neq j$

ADD $r0,r1,r2$;

B Exit;go to exit

Else :SUB $r0,r1,r2$;

Exit:

Instructions for making decisions

- If($i=j$), $f=g+h$; else $f=g-h$; in MIPS

beq reg1,reg2,L1 #go to statement L1 if reg1 ==reg2

bne reg1,reg2,L1 #go to statement L1 if reg1 $\neq$ reg2

f,g,h,i,j corresponds to five registers \$s0-\$s4

bne \$s3,\$s4,Else #go to Else if i $\neq$ j

add \$s0,\$s1,\$s2 # f=g+h skipped if i $\neq$ j

J Exit # go to exit #unconditional branch

Else:sub \$s0,\$s2,\$s1 # f=g-h skipped if i==j

Exit:

ARM and MIPS assembly for the loop

In ARM

while (save[i]==k)

i+=1;

Assume i and k corresponds to registers r3 and r5 and the base of the array save in r6

First step:save[i] in temp reg.before that add i to the base of the array *save* to form the address,multiply the index i by 4 due to the byte addressing problem.

Use LSL since shifting left 2 bits means multiply by 2^2

Loop:ADD r12,r6,r3,LSL #2 ;r12=address of save[i]

LDR r0,[r12,#0] ; temp reg r0=save[i]

CMP r0,r5

BNE Exit ;go to Exit if save[i] $\neq$ k

ADD r3,r3,#1 ;i=i+1

B loop ;go to loop

Exit;

In MIPS

Loop:sll \$t1,\$s3,2	;temp reg \$t1=4*i
add \$t1,\$t1,\$s6	;\$t1=address of save[i]
lw \$t0,0(\$t1)	;temp reg \$t0=save[i]
bne \$t0,\$s5,Exit	;go to exit if save[i]≠k
add \$s3,\$s3,1	;i=i+1
J loop	

Exit:

Example:

Let r0=1111 1111 and r1=0000 0001

CMP r0,r1

Which conditional branch is taken?

BLO L1 ;unsigned branch(branch on lower unsigned instruction is not taken to L1,since decimal of 1111 1111>1

BLT L2 ;signed branch(the branch on less than instruction is taken to L2,since in decimal -1<1

Note:an unsigned comparison of $x < y$ also checks if x is negative as well as if x is less than y

In MIPS the instruction `slt`(set on less than) used in *for* loops

`slt $t0,$s3,$s4` ;means register `$t0` is set to 1 if the `$s3<$s4`
otherwise `$t0` set to 0.

`slti $t0,$s2,10` ;`$t0=1` if `$s2<10`

The MIPS architecture does not include branch on less than because it is too complicated;either it would stretch the clock cycle time or it would take extra clock cycles per instruction.

MIPS compilers use `slt`,`slti`,`bne`,`beq` and the fixed value 0(`$zero`) to create all relative conditions,equal,not equal,less than,less than or equal,greater than,greater than or equal.

Case/Switch statements

- The simplest way to implement *switch* via a sequence of conditional tests,turning the *switch* statement into a chain of *if-then-else* statements.
- Sometimes the alternatives may be more efficiently encoded as a table of addresses of alternative instruction sequences,called a ***jump address table/jump table***.And the program needs only to index into the table and then jump to the appropriate sequence.
- The ***jump table*** is then just an array of words containing addresses that correspond to labels in the code.The program need to jump using the address in the appropriate entry from the jump table.
- ARM handles such situations implicitly by ***stored program*** concept .
- Need to have a register to hold the address of the current instruction being executed (***PC-program counter***)
- In ARM register 15 is the PC(also instruction address register)
- Any instruction with register 15 as destination register is an unconditional branch to the address at the value.

- Encoding branch instruction in ARM

Cond	12	address
4bits	4bits	24bits

- The cond field encodes the many versions of the conditional branch like

Value	Meaning	Value	Meaning
0	EQ(Equal)	8	HI(unsigned higher)
1	NE(Not Equal)	9	LS(unsigned Lower or Same)
2	HS(unsigned Higher or Same)	10	GE(signed Greater than or Equal)
3	LO(unsigned LOWER)	11	LT(signed Less Than)
4	MI(Minus,<0)	12	GT(signed Greater Than)
5	PL(PLus,>=0)	13	LE(signed Less Than or Equal)
6	VS(oVerflow Set,overflow)	14	AL(ALways)
7	VC(oVerflow Clear,no overflow)	15	NV(reserved)

- The cond field encodes the many versions of the conditional branch (shown in the table)
- The 24 bit address limit the programs to 2^{24} or 16MB which would be fine for many programs but constrains large ones.
- An alternative would be a register that added to branch address,so the branch instruction would calculate $PC = \text{Reg} + \text{branch address}$.
- The sum would allow the programs to be as large as 2^{32} ,solving the branch address size problem.'
- Which is that register?
- Conditional branches are found in loops and in *if* statements,so they tend to branch to a nearby instruction.Ex about half of all conditional branches in SPEC benchmarks go to locations less than 16 instructions away.
- Since the PC contains the address of the current instruction,we can branch within $\pm 2^{24}$ words of the current instruction if we use the PC as the register to be added to the address.All the loops and *if* statements are much smaller than $\pm 2^{24}$ words,so the PC is the ideal choice.
- Also called as PC-relative addressing.

Conditional Execution

- Unusual feature of ARM is most instruction can be conditionally executed ,not just branches.This is the purpose of the 4 bit cond field found in the most ARM instruction formats.
- The assembly language programmer simply appends the desired condition to the instruction name to perform the operation only if the condition is true based on the last time the condition flags were set.

- Ex

CMP r3,r4

ADDEQ r0,r1,r2

SUBNE r0,r1,r2

Procedures in Computer Hardware

- **Procedure:** A stored subroutine that performs a specific task based on the parameters with which it is provided.
- Registers are the fastest place to hold data in a computer, so use them as much as possible. ARM s/w follows the following conventions for procedure calling in allocating 16 registers.
- **r0-r3** (four argument registers in which to pass parameters).
- **lr**: one link register containing the return address register to return to the point of origin.
- **BL**: It jumps to an address and simultaneously save the address of the following instruction in register lr. (**Branch-and-link instruction**).
- **BL: ProcedureAddress.**
- (**BL instruction that jumps to an address and simultaneously and saves the address of the following instruction in a register (lr or register 14).**

- MOV pc,lr
- The calling program ,or caller,puts the parameter values in r0-r3 and uses BL X to jump to procedure X(callee).The callee then performs the calculations, places the results (if any) into r0 and r1,and returns control to the caller using MOV pc,lr.
- The BL instruction actually saves PC+4 in register lr to link to the following instruction to set up the procedure return.
- **Stack:**
- Suppose a compiler needs more registers for a procedure than the four arguments and two return value registers.Since we must cover our tracks after our mission is complete,

any registers needed by the caller must be restored to the values that they contained ***before*** the procedure was invoked.This situation is an example in which we need to spill registers to memory.

The ideal data structure for spilling registers is a stack –a LIFO queue

- A stack needs a pointer to the most recently allocated address in the stack to show where the next procedure should place the registers to be spilled or where old register values are found.
- The stack pointer is adjusted by one word for each register that is saved or restored. ARM reserves register 13 for SP.
- Compile the C procedure that does not call another procedure

```
int example(int g,int h,int i, int j)
{
    int f;
    f=(g+h)-(i+j)
    return f;
}
```

- Let g,h,i,j correspond to the argument registers r0,r1,r2,r3 and f corresponds to r4.
- Label of the procedure is ex_procedure:
- Save three registers:r4,r5,r6.
- “push” the old values onto the stack by creating space for three words(12 bytes) on the stack and then store them:

SUB sp,sp,#12 :adjust stack to make room for 3 items

STR r6,[sp,#8] :save register r6 for use afterwords

STR r5,[sp,#4] :save register r5 for use afterwords

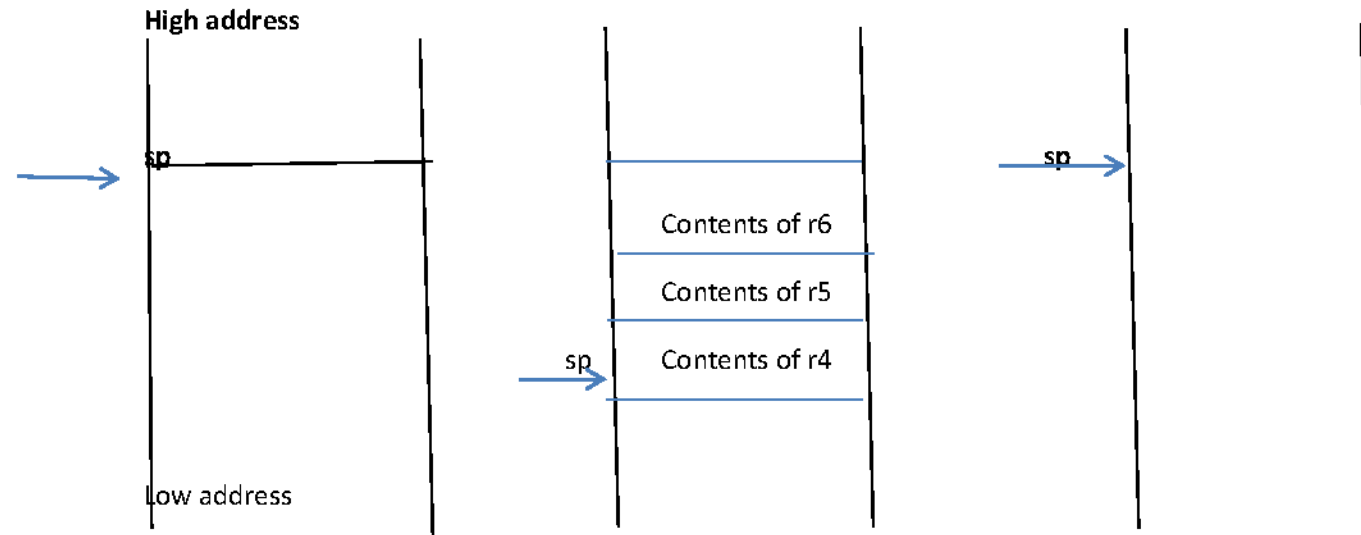
STR r4,[sp,#0] :save register r4 for use afterwords

ADD r5,r0,r1 :reg r5 contains g+h

ADD r6,r2,r3 :reg r6 contains i+j

SUB r4,r5,r6 :f gets r5-r6,which is (g+h)-(i+j)

Values of sp and the stack: before, during and after the procedure call



To return the value of f we can copy it into a return value register r0:

```
MOV r0,r4 ; return f(r0=r4)
```

Before returning ,we restore the three old values of the registers we saved by “popping” them from the stack.

```
LDR r4,[sp,#0] ;restore register r4 for caller
```

```
LDR r5,[sp,#4] ;restore register r5 for caller
```

```
LDR r6,[sp,#8] ;restore register r6 for caller
```

```
ADD sp,sp,#12 ;adjust stack to delete 3 items
```

The procedure ends with a jump register using the return address:

```
MOV pc,lr ;jump back to calling routine.
```

We used temporary registers and assumed their old values must be saved and restored.To avoid saving and restoring a register whose value is never used,which might happen with a temporary register,ARM software separates 12 of the registers into two groups:

r0-r3,r12:argument or scratch registers that are not preserved by the callee(called procedure)on a procedure call

r4-r11:eight variable registers that must be preserved on a procedure call(if used ,the callee saves and restores them)

Note:This simple convention reduces register spilling .In the example above,if we could rewrite the code to use r12 and reuse one of the r0 to r3,we can drop two stores and two loads from the code. We still must save and restore r4,since the callee must assume that the caller needs its value.

In MIPS:

MIPS s/w follows the following convention in allocating its 32 registers for procedure calling:

\$a0-\$a3 : four argument register in which to pass parameters

\$v0-\$v1: two value registers in which to return values.

\$ra : one return address register to return to the point of origin.

jal: jump-and-link instruction(an instruction that jumps to an address and simultaneously saves the address of the following instruction in a register(\$ra in MIPS)

jal ProcedureAddress

jal instruction saves PC+4 in register \$ra to link to the following instruction to set up the procedure return.

jr \$ra ;jump register instruction meaning an unconditional jump to the address specified in a register.

The calling program ,or caller,puts the parameter values in \$a0-\$a3 and uses jal X to jump to procedure X(callee).The callee then performs the calculations, places the results (if any) into \$v0 and \$v1,and returns control to the caller using jr \$ra.

- Let g,h,i,j correspond to the argument registers \$a0,\$a1,\$a2,\$a3 and f corresponds to \$s0.
- Label of the procedure is ex_procedure:
- Save three registers:\$s0,\$t0,\$t1.
- “push” the old values onto the stack by creating space for three words(12 bytes) on the stack and then store them:

```

addi $sp,$sp,-12 :adjust stack to make room for 3 items
sw  $t1,8[$sp]   :save register $t1 for use afterwords
sw  $t0,4[$sp]   :save register $t0 for use afterwords
sw  $s0,0[$sp]   :save register $s0 for use afterwords
add $t0,$a0,$a1  :reg $t0 contains g+h
add $t1,$a2,$a3  :reg $t1 contains i+j
sub $s0,$t0,$t1  :f gets $t0-$t1,which is (g+h)-(i+j)
add $v0,$s0,$zero :returns f ($v0=$s0+0)

```

Before returning ,we restore the three old values of the registers we saved by “popping” them from the stack.

```
lw $s0,0($sp)    ;restore register $s0 for caller
lw $t0,4($sp)    ;restore register $t0 for caller
lw $t1,8($sp)    ;restore register $t1 for caller
addi sp,sp,12     ;adjust stack to delete 3 items
```

The procedure ends with a jump register using the return address:

```
jr $ra           ;jump back to calling routine.
```

We used temporary registers and assumed their old values must be saved and restored.To avoid saving and restoring a register whose value is never used,which might happen with a temporary register,MIPS software separates 18 of the registers into two groups:

\$t0-\$t9 :10 temp registers that are not preserved by the callee(called procedure)on a procedure call

\$s0-\$s7:eight saved registers that must be preserved on a procedure call(if used ,the callee saves and restores them)

Note: This simple convention reduces register spilling. In the example above, since the caller (procedure doing the calling) does not expect registers \$t0 and \$t1 to be preserved across a procedure call, we can drop two stores and two loads from the code. We still must save and restore \$s0, since the callee must assume that the caller needs its value.

Nested Procedures:

Ex. Suppose that the main program calls procedure A with an argument of 3, by placing the value 3 into register r0 and then using BL A. Then suppose that procedure A calls procedure B via BL B with an argument of 7, also placed in r0. Since A has not finished its task yet, there is a conflict over the use of register r0. Similarly there is a conflict over the return address in register lr, since it now has the return address for B.

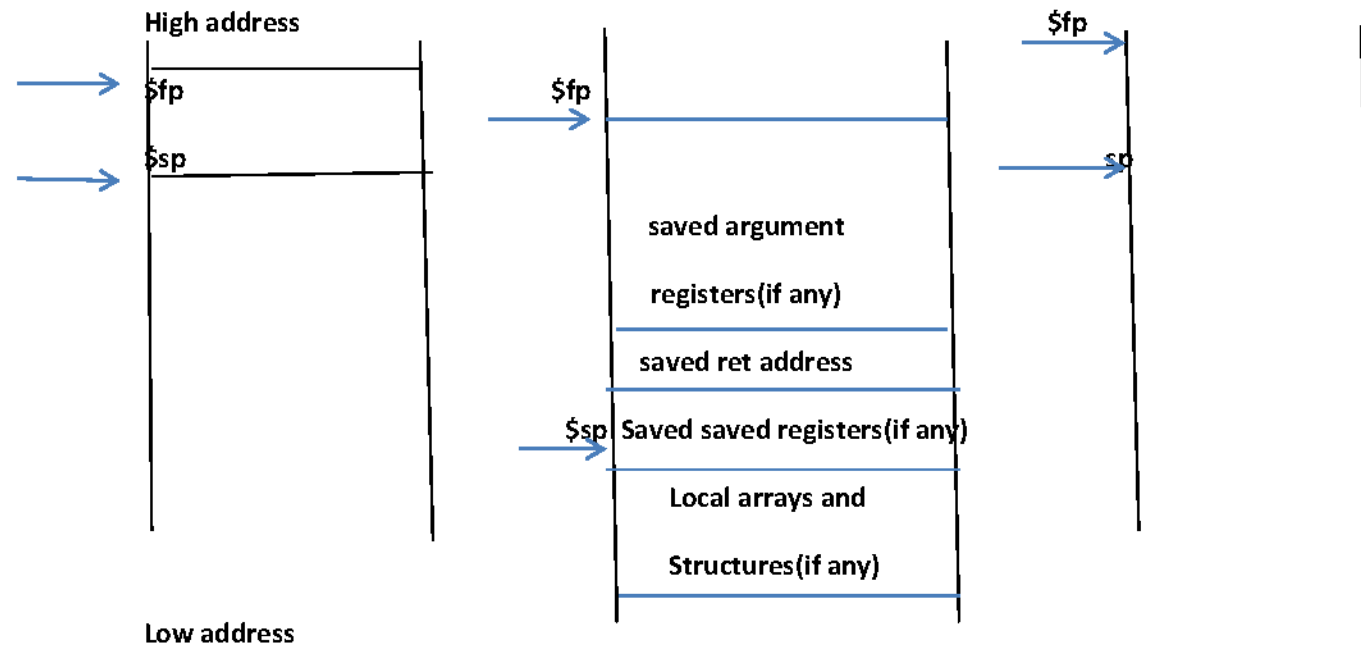
Solution: push all the other registers that must be preserved onto the stack.

- The caller pushes any argument registers(r0-r3) that are needed after the call. The callee pushes the return address register lr and any variable registers (r4-r11)used by the callee. The sp is adjusted to account for the number of registers placed on the stack.Upon the return, the registers are restored from memory and the sp is readjusted.
- Convert into into ARM assembly code?

```
int fact(int n)
{
if(n<1)return (1);
else return(n*fact(n-1));
}
```

fact:ARM	fact:MIPS
SUB sp,sp,#8 ;adjust stack for 2 items	addi sp,sp,-8;adjust stack for 2 items
STR lr ,[sp,#4] ;save return address	sw \$ra,4(\$sp) ;save return address
STR r0,[sp,#0] ;save the argument n	sw \$a0,0(\$sp) ;save the argument n
CMP r0,#1 ;compare n to 1	slti \$t0,\$a0,1;test for n<1
BGE L1 ;if n>=1,go to L1	beq \$t0,\$zero,L1;if n>=1,go to L1
MOV r0,#1 ;return 1	addi \$v0,\$zero,1 ;return 1
ADD sp,sp,#8 ;pop two items off stack	addi \$sp,\$sp,#8;pop two items off stack
MOV pc,lr ;return to the caller	jr \$ra ;return to after jal
L1: SUB r0,r0,#1 ;n>=1:argument gets(n-1)	L1:addi \$a0,\$a0,-1; n>=1:argument gets(n-1)
BL fact ;call fact with (n-1)	jal fact ; call fact with (n-1)
MOV r12,r0 ;save the return value	lw \$a0,0(\$sp); return from jal;restore argument n
LDR r0,[sp,#0] ;return from BL;restore argument n	lw \$ra,4(\$sp); restore the return address
LDR lr [sp,#0] ;restore the return address	addi \$sp,\$sp,8 ;adjust sp to pop 2 items
ADD sp,sp,#8 ;adjust sp to pop 2 items	mul \$v0,\$a0,\$v0; return n * fact(n-1)
MUL r0,r0,r12 ;return n * fact(n-1)	jr \$ra ; return to caller

Stack allocation before during and after procedural call

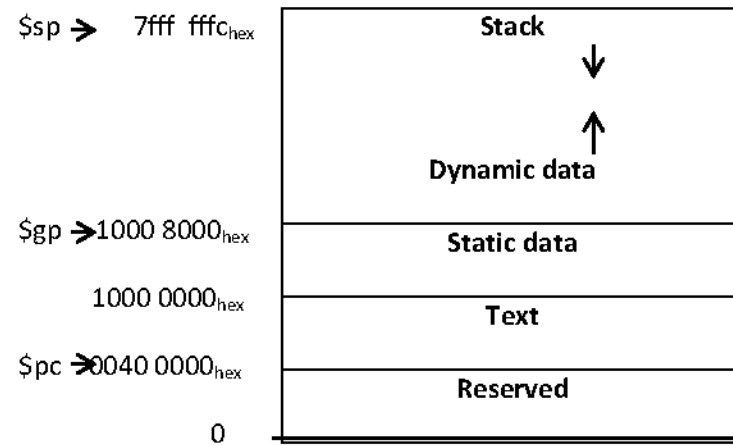


What is and what not preserved across a procedure call

Preserved	Not Preserved
Variable registers:r4-r11	Argument registers:r0-r3
Stack pointer register:sp	Intra procedure-call scratch register:r12
Link register:lr	Stack below the sp
Stack above the sp	

- Storage classes in C: automatic (local to procedure) and static (variables outside procedures).
- Global pointer (\$gp): To simplify access to global data MIPS s/w reserves another register called global pointer.
- Allocating space for new data on stack: The final complexity is that stack is also used to store variables that are local to the procedure that do not fit in the registers such as local arrays and structures.
- The segment of the stack containing a procedure's saved registers and local variables is called : **procedure frame (activation record)**.
- **Some MIPS s/w uses a frame pointer (\$fp) to point to first word of the frame of the procedure.**
- **\$fp is the value denoting the location of the saved registers and local variables for a given procedure.**
- **\$sp points to the top of the stack. when a \$fp is used, it is initialized using the address in \$sp on a call and \$sp is restored using \$sp.**

Allocating space for new data on the heap, MIPS convention



- In addition to the automatic variables that are local to procedures, C programmers need space in memory for static variables and for dynamic data structures.
- Text segment: Segment of the unix object file that contains the m/c code for routines in the source file.
- Static data segment: place for constants and other static variables.
- Data structures like linked lists tend to grow and shrink during their lifetimes. The segments for such DSs is the *heap*.
- ***Stack and heap grow towards each other allowing efficient use of memory as two segments wax and wane.***
- ***C allocates and free space on the heap with explicit functions, malloc() allocates space on the heap and returns a pointer to it and free() releases space on the heap to which the pointer points.***

- Memory allocation is controlled by programs in C, and it is the source of many common and difficult bugs.
- Forgetting to free space leads to “**memory leak**”, which eventually uses up so much memory that the OS may crash. Freeing space too early leads to “**dangling pointers**”, which can cause pointers to point to things that the program never intended.
- **GNU MIPS C compiler uses a frame pointer. C compiler from MIPS/Silicon graphics does not use fp, it uses register 30 as another save register.**